JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class Locale

java.lang.Object
    java.util.Locale

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public final class Locale
extends Object
implements Cloneable, Serializable
```

A `Locale` object represents a specific geographical, political, or cultural region. An operation that requires a `Locale` to perform its task is called *locale-sensitive* and uses the `Locale` to tailor information for the user. For example, displaying a number is a locale-sensitive operation— the number should be formatted according to the customs and conventions of the user's native country, region, or culture.

The `Locale` class implements IETF BCP 47 which is composed of RFC 4647 "Matching of Language Tags" and RFC 5646 "Tags for Identifying Languages" with support for the LDML (UTS#35, "Unicode Locale Data Markup Language") BCP 47-compatible extensions for locale data exchange.

A `Locale` object logically consists of the fields described below.

language

ISO 639 alpha-2 or alpha-3 language code, or registered language subtags up to 8 alpha letters (for future enhancements). When a language has both an alpha-2 code and an alpha-3 code, the alpha-2 code must be used. You can find a full list of valid language codes in the IANA Language Subtag Registry (search for "Type: language"). The language field is case insensitive, but Locale always canonicalizes to lower case.

Well-formed language values have the form [a-zA-Z]{2,8}. Note that this is not the the full BCP47 language production, since it excludes extlang. They are not needed since modern three-letter language codes replace them.

Example: "en" (English), "ja" (Japanese), "kok" (Konkani)

script

ISO 15924 alpha-4 script code. You can find a full list of valid script codes in the IANA Language Subtag Registry (search for "Type: script"). The script field is case insensitive, but Locale always canonicalizes to title case (the first letter is upper case and the rest of the letters are lower case).

Well-formed script values have the form [a-zA-Z]{4}

Example: "Latn" (Latin), "Cyrl" (Cyrillic)

country (region)

ISO 3166 alpha-2 country code or UN M.49 numeric-3 area code. You can find a full list of valid country and region codes in the IANA Language Subtag Registry (search for "Type: region"). The country (region) field is case insensitive, but Locale always canonicalizes to upper case.

Well-formed country/region values have the form [a-zA-Z]{2} | [0-9]{3}

Example: "US" (United States), "FR" (France), "029" (Caribbean)

variant

Any arbitrary value used to indicate a variation of a Locale. Where there are two or more variant values each indicating its own semantics, these values should be ordered by importance, with most important first, separated by underscore('_'). The variant field is case sensitive.

Note: IETF BCP 47 places syntactic restrictions on variant subtags. Also BCP 47 subtags are strictly used to indicate additional variations that define a language or its dialects that are not covered by any combinations of language, script and region subtags. You can find a full list of valid variant codes in the IANA Language Subtag Registry (search for "Type: variant").

However, the variant field in Locale has historically been used for any kind of variation, not just language variations. For example, some supported variants available in Java SE Runtime Environments indicate alternative cultural behaviors such as calendar type or number script. In BCP 47 this kind of information, which does not identify the language, is supported by extension subtags or private use subtags.

Well-formed variant values have the form SUBTAG (('_'|'-') SUBTAG)* where SUBTAG = [0-9][0-9a-zA-Z]{3} | [0-9a-zA-Z]{5,8}. (Note: BCP 47 only uses hyphen ('-') as a delimiter, this is more lenient).

Example: "polyton" (Polytonic Greek), "POSIX"

extensions

A map from single character keys to string values, indicating extensions apart from language identification. The extensions in Locale implement the semantics and syntax of BCP 47 extension subtags and private use subtags. The extensions are case insensitive, but Locale canonicalizes all extension keys and values to lower case. Note that extensions cannot have empty values.

Well-formed keys are single characters from the set [0-9a-zA-Z]. Well-formed values have the form SUBTAG ('-' SUBTAG)* where for the key 'x' SUBTAG = [0-9a-zA-Z]{1,8} and for other keys SUBTAG = [0-9a-zA-Z]{2,8} (that is, 'x' allows single-character subtags).

Example: key="u"/value="ca-japanese" (Japanese Calendar), key="x"/value="java-1-7"

Note: Although BCP 47 requires field values to be registered in the IANA Language Subtag Registry, the `Locale` class does not provide any validation features. The `Builder` only checks if an individual field satisfies the syntactic requirement (is well-formed), but does not validate the value itself. See `Locale.Builder` for details.

Unicode locale/language extension

UTS#35, "Unicode Locale Data Markup Language" defines optional attributes and keywords to override or refine the default behavior associated with a locale. A keyword is represented by a pair of key and type. For example, "nu-thai" indicates that Thai local digits (value:"thai") should be used for formatting numbers (key:"nu").

The keywords are mapped to a BCP 47 extension value using the extension key 'u' (`UNICODE_LOCALE_EXTENSION`). The above example, "nu-thai", becomes the extension "u-nu-thai".code

Thus, when a `Locale` object contains Unicode locale attributes and keywords, `getExtension(UNICODE_LOCALE_EXTENSION)` will return a String representing this information, for example, "nu-thai". The `Locale` class also provides `getUnicodeLocaleAttributes()`, `getUnicodeLocaleKeys()`, and `getUnicodeLocaleType(java.lang.String)` which allow you to access Unicode locale attributes and key/type pairs directly. When represented as a string, the Unicode Locale Extension lists attributes alphabetically, followed by key/type sequences with keys listed alphabetically (the order of subtags comprising a key's type is fixed when the type is defined)

A well-formed locale key has the form `[0-9a-zA-Z]{2}`. A well-formed locale type has the form `"" | [0-9a-zA-Z]{3,8} ('-' [0-9a-zA-Z]{3,8})*` (it can be empty, or a series of subtags 3-8 alphanums in length). A well-formed locale attribute has the form `[0-9a-zA-Z]{3,8}` (it is a single subtag with the same form as a locale type subtag).

The Unicode locale extension specifies optional behavior in locale-sensitive services. Although the LDML specification defines various keys and values, actual locale-sensitive service implementations in a Java Runtime Environment might not support any particular Unicode locale attributes or key/type pairs.

Creating a Locale

There are several different ways to create a `Locale` object.

Builder

Using `Locale.Builder` you can construct a `Locale` object that conforms to BCP 47 syntax.

Constructors

The `Locale` class provides three constructors:

```
Locale(String language)
Locale(String language, String country)
Locale(String language, String country, String variant)
```

These constructors allow you to create a `Locale` object with language, country and variant, but you cannot specify script or extensions.

Factory Methods

The method `forLanguageTag(java.lang.String)` creates a `Locale` object for a well-formed BCP 47 language tag.

Locale Constants

The `Locale` class provides a number of convenient constants that you can use to create `Locale` objects for commonly used locales. For example, the following creates a `Locale` object for the United States:

```
Locale.US
```

Locale Matching

If an application or a system is internationalized and provides localized resources for multiple locales, it sometimes needs to find one or more locales (or language tags) which meet each user's specific preferences. Note that a term "language tag" is used interchangeably with "locale" in this locale matching documentation.

In order to do matching a user's preferred locales to a set of language tags, RFC 4647 Matching of Language Tags defines two mechanisms: filtering and lookup. *Filtering* is used to get all matching locales, whereas *lookup* is to choose the best

matching locale. Matching is done case-insensitively. These matching mechanisms are described in the following sections.

A user's preference is called a *Language Priority List* and is expressed as a list of language ranges. There are syntactically two types of language ranges: basic and extended. See `Locale.LanguageRange` for details.

Filtering

The filtering operation returns all matching language tags. It is defined in RFC 4647 as follows: "In filtering, each language range represents the least specific language tag (that is, the language tag with fewest number of subtags) that is an acceptable match. All of the language tags in the matching set of tags will have an equal or greater number of subtags than the language range. Every non-wildcard subtag in the language range will appear in every one of the matching language tags."

There are two types of filtering: filtering for basic language ranges (called "basic filtering") and filtering for extended language ranges (called "extended filtering"). They may return different results by what kind of language ranges are included in the given Language Priority List. `Locale.FilteringMode` is a parameter to specify how filtering should be done.

Lookup

The lookup operation returns the best matching language tags. It is defined in RFC 4647 as follows: "By contrast with filtering, each language range represents the most specific tag that is an acceptable match. The first matching tag found, according to the user's priority, is considered the closest match and is the item returned."

For example, if a Language Priority List consists of two language ranges, `"zh-Hant-TW"` and `"en-US"`, in prioritized order, lookup method progressively searches the language tags below in order to find the best matching language tag.

1. `zh-Hant-TW`
2. `zh-Hant`
3. `zh`
4. `en-US`
5. `en`

If there is a language tag which matches completely to a language range above, the language tag is returned.

`"*"` is the special language range, and it is ignored in lookup.

If multiple language tags match as a result of the subtag `'*'` included in a language range, the first matching language tag returned by an `Iterator` over a `Collection` of language tags is treated as the best matching one.

Use of Locale

Once you've created a `Locale` you can query it for information about itself. Use `getCountry` to get the country (or region) code and `getLanguage` to get the language code. You can use `getDisplayCountry` to get the name of the country suitable for displaying to the user. Similarly, you can use `getDisplayLanguage` to get the name of the language suitable for displaying to the user. Interestingly, the `getDisplayXXX` methods are themselves locale-sensitive and have two versions: one that uses the default `DISPLAY` locale and one that uses the locale specified as an argument.

The Java Platform provides a number of classes that perform locale-sensitive operations. For example, the `NumberFormat` class formats numbers, currency, and percentages in a locale-sensitive manner. Classes such as `NumberFormat` have several convenience methods for creating a default object of that type. For example, the `NumberFormat` class provides these three convenience methods for creating a default `NumberFormat` object:

```
NumberFormat.getInstance()
NumberFormat.getCurrencyInstance()
NumberFormat.getPercentInstance()
```

Each of these methods has two variants; one with an explicit locale and one without; the latter uses the default `FORMAT` locale:

```
NumberFormat.getInstance(myLocale)
NumberFormat.getCurrencyInstance(myLocale)
NumberFormat.getPercentInstance(myLocale)
```

A `Locale` is the mechanism for identifying the kind of object (`NumberFormat`) that you would like to get. The locale is **just** a mechanism for identifying objects, **not** a container for the objects themselves.

Compatibility

In order to maintain compatibility with existing usage, Locale's constructors retain their behavior prior to the Java Runtime Environment version 1.7. The same is largely true for the `toString` method. Thus Locale objects can continue to be used as they were. In particular, clients who parse the output of toString into language, country, and variant fields can continue to do so (although this is strongly discouraged), although the variant field will have additional information in it if script or extensions are present.

In addition, BCP 47 imposes syntax restrictions that are not imposed by Locale's constructors. This means that conversions between some Locales and BCP 47 language tags cannot be made without losing information. Thus `toLanguageTag` cannot represent the state of locales whose language, country, or variant do not conform to BCP 47.

Because of these issues, it is recommended that clients migrate away from constructing non-conforming locales and use the `forLanguageTag` and `Locale.Builder` APIs instead. Clients desiring a string representation of the complete locale can then always rely on `toLanguageTag` for this purpose.

Special cases

For compatibility reasons, two non-conforming locales are treated as special cases. These are **`ja_JP_JP`** and **`th_TH_TH`**. These are ill-formed in BCP 47 since the variants are too short. To ease migration to BCP 47, these are treated specially during construction. These two cases (and only these) cause a constructor to generate an extension, all other values behave exactly as they did prior to Java 7.

Java has used `ja_JP_JP` to represent Japanese as used in Japan together with the Japanese Imperial calendar. This is now representable using a Unicode locale extension, by specifying the Unicode locale key `ca` (for "calendar") and type `japanese`. When the Locale constructor is called with the arguments "ja", "JP", "JP", the extension "u-ca-japanese" is automatically added.

Java has used `th_TH_TH` to represent Thai as used in Thailand together with Thai digits. This is also now representable using a Unicode locale extension, by specifying the Unicode locale key `nu` (for "number") and value `thai`. When the Locale constructor is called with the arguments "th", "TH", "TH", the extension "u-nu-thai" is automatically added.

Serialization

During serialization, writeObject writes all fields to the output stream, including extensions.

During deserialization, readResolve adds extensions as described in Special Cases, only for the two cases th_TH_TH and ja_JP_JP.

Legacy language codes

Locale's constructor has always converted three language codes to their earlier, obsoleted forms: `he` maps to `iw`, `yi` maps to `ji`, and `id` maps to `in`. This continues to be the case, in order to not break backwards compatibility.

The APIs added in 1.7 map between the old and new language codes, maintaining the old codes internal to Locale (so that `getLanguage` and `toString` reflect the old code), but using the new codes in the BCP 47 language tag APIs (so that `toLanguageTag` reflects the new one). This preserves the equivalence between Locales no matter which code or API is used to construct them. Java's default resource bundle lookup mechanism also implements this mapping, so that resources can be named using either convention, see `ResourceBundle.Control`.

Three-letter language/country(region) codes

The Locale constructors have always specified that the language and the country param be two characters in length, although in practice they have accepted any

length. The specification has now been relaxed to allow language codes of two to eight characters and country (region) codes of two to three characters, and in particular, three-letter language codes and three-digit region codes as specified in the IANA Language Subtag Registry. For compatibility, the implementation still does not impose a length constraint.

Since:

1.1

See Also:

`Locale.Builder`, `ResourceBundle`, `Format`, `NumberFormat`, `Collator`, `Serialized Form`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	**`Locale.Builder`**	`Builder` is used to build instances of `Locale` from values configured by the setters.
static class	**`Locale.Category`**	Enum for locale categories.
static class	**`Locale.FilteringMode`**	This enum provides constants to select a filtering mode for locale matching.
static class	**`Locale.LanguageRange`**	This class expresses a *Language Range* defined in **RFC 4647 Matching of Language Tags**.

Field Summary

Fields

Modifier and Type	Field	Description
static **`Locale`**	**`CANADA`**	Useful constant for country.
static **`Locale`**	**`CANADA_FRENCH`**	Useful constant for country.
static **`Locale`**	**`CHINA`**	

		Useful constant for country.
static **Locale**	**CHINESE**	Useful constant for language.
static **Locale**	**ENGLISH**	Useful constant for language.
static **Locale**	**FRANCE**	Useful constant for country.
static **Locale**	**FRENCH**	Useful constant for language.
static **Locale**	**GERMAN**	Useful constant for language.
static **Locale**	**GERMANY**	Useful constant for country.
static **Locale**	**ITALIAN**	Useful constant for language.
static **Locale**	**ITALY**	Useful constant for country.
static **Locale**	**JAPAN**	Useful constant for country.
static **Locale**	**JAPANESE**	Useful constant for language.
static **Locale**	**KOREA**	Useful constant for country.
static **Locale**	**KOREAN**	Useful constant for language.
static **Locale**	**PRC**	Useful constant for country.
static char	**PRIVATE_USE_EXTENSION**	The key for the private use extension ('x').
static **Locale**	**ROOT**	Useful constant for the root locale.
static **Locale**	**SIMPLIFIED_CHINESE**	Useful constant for language.

static Locale	TAIWAN	Useful constant for country.
static Locale	TRADITIONAL_CHINESE	Useful constant for language.
static Locale	UK	Useful constant for country.
static char	UNICODE_LOCALE_EXTENSION	The key for Unicode locale extension ('u').
static Locale	US	Useful constant for country.

Constructor Summary

Constructors

Constructor	Description
Locale(String language)	Construct a locale from a language code.
Locale(String language, String country)	Construct a locale from language and country.
Locale(String language, String country, String variant)	Construct a locale from language, country and variant.

Method Summary

Concrete Methods

Modifier and Type	Method	Descripti
Object	clone()	Override Cloneabl
boolean	equals(Object obj)	Returns this Loca equal to another object.
static List<Locale>	filter(List<Locale.LanguageRange> priorityList, Collection<Locale> locales)	Returns of match Locale instance using the filtering mechani defined i RFC 464
static List<Locale>	filter(List<Locale.LanguageRange> priorityList, Collection<Locale> locales, Locale.FilteringMode mode)	Returns of match Locale instance using the filtering mechani defined i RFC 464
static List<String>	filterTags(List<Locale.LanguageRange> priorityList, Collection<String> tags)	Returns of match language tags usin basic filt mechani defined i RFC 464
static List<String>	filterTags(List<Locale.LanguageRange> priorityList, Collection<String> tags, Locale.FilteringMode mode)	Returns of match language tags usin basic filt mechani defined i RFC 464

static **Locale**	**forLanguageTag**(**String** languageTag)	Returns locale fo specified BCP 47 language string.
static **Locale**[]	**getAvailableLocales**()	Returns array of installed locales.
String	**getCountry**()	Returns country/ region co for this l which sh either be empty st an upper ISO 3166 letter co a UN M. digit cod
static **Locale**	**getDefault**()	Gets the current v of the de locale fo instance the Java Virtual Machine
static **Locale**	**getDefault**(**Locale.Category** category)	Gets the current v of the de locale fo specified Category this insta of the Ja Virtual Machine
String	**getDisplayCountry**()	Returns name for locale's country

		appropri for displ the user.
String	getDisplayCountry(Locale inLocale)	Returns name for locale's country appropri for displ the user.
String	getDisplayLanguage()	Returns name for locale's language is appro for displ the user.
String	getDisplayLanguage(Locale inLocale)	Returns name for locale's language is appro for displ the user.
String	getDisplayName()	Returns name for locale th appropri for displ the user.
String	getDisplayName(Locale inLocale)	Returns name for locale th appropri for displ the user.
String	getDisplayScript()	Returns name for the local script th appropri for displ the user.

String	getDisplayScript(Locale inLocale)	Returns name for locale's s that is appropri for displa the user.
String	getDisplayVariant()	Returns name for locale's variant c that is appropri for displa the user.
String	getDisplayVariant(Locale inLocale)	Returns name for locale's variant c that is appropri for displa the user.
String	getExtension(char key)	Returns extensio private u value associate with the specified or null if is no extensio associate with the
Set<Character>	getExtensionKeys()	Returns set of extensio associate with this locale, o empty se has no extensio

String	getISO3Country()	Returns three-let abbrevia for this locale's country.
String	getISO3Language()	Returns three-let abbrevia of this lo language
static String[]	getISOCountries()	Returns of all 2-l country defined i 3166.
static String[]	getISOLanguages()	Returns of all 2-l language codes de in ISO 6
String	getLanguage()	Returns language of this L
String	getScript()	Returns script fo locale, w should e be the e string or ISO 159 letter sc code.
Set<String>	getUnicodeLocaleAttributes()	Returns set of un locale attribute associate with this locale, o empty se has no attribute

Set<String>	getUnicodeLocaleKeys()	Returns set of Un locale ke defined l locale, o empty se this local none.
String	getUnicodeLocaleType(String key)	Returns Unicode locale ty associate with the specified Unicode locale ke this local
String	getVariant()	Returns variant c for this l
boolean	hasExtensions()	Returns if this Lo has any extensio
int	hashCode()	Override hashCod
static Locale	lookup(List<Locale.LanguageRange> priorityList, Collection<Locale> locales)	Returns Locale instance the best-matching language using the lookup mechani defined i RFC 464
static String	lookupTag(List<Locale.LanguageRange> priorityList, Collection<String> tags)	Returns best-mat language using the lookup mechani

		defined i RFC 464
static void	**setDefault**(**Locale.Category** category, **Locale** newLocale)	Sets the default l for the specified Category this insta of the Ja Virtual Machine
static void	**setDefault**(**Locale** newLocale)	Sets the default l for this instance the Java Virtual Machine
Locale	**stripExtensions**()	Returns copy of t Locale v no **extensio**
String	**toLanguageTag**()	Returns well-form IETF BC language represen this loca
String	**toString**()	Returns string represen of this Lo object, consistin language country, variant, and extensio below: language "_" + cou + "_" + (variant

		"_#" | "# script + extensio Languag always lo case, cou is always upper ca script is always ti case, and extensio always lo case.

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

ENGLISH

```
public static final Locale ENGLISH
```

Useful constant for language.

FRENCH

```
public static final Locale FRENCH
```

Useful constant for language.

GERMAN

```
public static final Locale GERMAN
```

Useful constant for language.

ITALIAN

```
public static final Locale ITALIAN
```

Useful constant for language.

JAPANESE

```
public static final Locale JAPANESE
```

Useful constant for language.

KOREAN

```
public static final Locale KOREAN
```

Useful constant for language.

CHINESE

```
public static final Locale CHINESE
```

Useful constant for language.

SIMPLIFIED_CHINESE

```
public static final Locale SIMPLIFIED_CHINESE
```

Useful constant for language.

TRADITIONAL_CHINESE

```
public static final Locale TRADITIONAL_CHINESE
```

Useful constant for language.

FRANCE

```
public static final Locale FRANCE
```

Useful constant for country.

GERMANY

```
public static final Locale GERMANY
```

Useful constant for country.

ITALY

`public static final Locale ITALY`

Useful constant for country.

JAPAN

`public static final Locale JAPAN`

Useful constant for country.

KOREA

`public static final Locale KOREA`

Useful constant for country.

CHINA

`public static final Locale CHINA`

Useful constant for country.

PRC

`public static final Locale PRC`

Useful constant for country.

TAIWAN

`public static final Locale TAIWAN`

Useful constant for country.

UK

`public static final Locale UK`

Useful constant for country.

US

```
public static final Locale US
```

Useful constant for country.

CANADA

```
public static final Locale CANADA
```

Useful constant for country.

CANADA_FRENCH

```
public static final Locale CANADA_FRENCH
```

Useful constant for country.

ROOT

```
public static final Locale ROOT
```

Useful constant for the root locale. The root locale is the locale whose language, country, and variant are empty ("") strings. This is regarded as the base locale of all locales, and is used as the language/country neutral locale for the locale sensitive operations.

Since:

1.6

PRIVATE_USE_EXTENSION

```
public static final char PRIVATE_USE_EXTENSION
```

The key for the private use extension ('x').

Since:

1.7

See Also:

getExtension(char), Locale.Builder.setExtension(char, String), Constant Field Values

UNICODE_LOCALE_EXTENSION

```
public static final char UNICODE_LOCALE_EXTENSION
```

The key for Unicode locale extension ('u').

Since:

1.7

See Also:

`getExtension(char)`, `Locale.Builder.setExtension(char, String)`, `Constant Field Values`

Constructor Detail

Locale

```
public Locale(String language,
              String country,
              String variant)
```

Construct a locale from language, country and variant. This constructor normalizes the language value to lowercase and the country value to uppercase.

Note:

- ISO 639 is not a stable standard; some of the language codes it defines (specifically "iw", "ji", and "in") have changed. This constructor accepts both the old codes ("iw", "ji", and "in") and the new codes ("he", "yi", and "id"), but all other API on Locale will return only the OLD codes.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.
- The two cases ("ja", "JP", "JP") and ("th", "TH", "TH") are handled specially, see Special Cases for more information.

Parameters:

`language - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the Locale class description about valid language values.`

`country - An ISO 3166 alpha-2 country code or a UN M.49 numeric-3 area code. See the Locale class description about valid country values.`

`variant - Any arbitrary value used to indicate a variation of a Locale. See the Locale class description for the details.`

Throws:

NullPointerException - thrown if any argument is null.

Locale

```
public Locale(String language,
              String country)
```

Construct a locale from language and country. This constructor normalizes the language value to lowercase and the country value to uppercase.

Note:

- ISO 639 is not a stable standard; some of the language codes it defines (specifically "iw", "ji", and "in") have changed. This constructor accepts both the old codes ("iw", "ji", and "in") and the new codes ("he", "yi", and "id"), but all other API on Locale will return only the OLD codes.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.

Parameters:

language - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the Locale class description about valid language values.

country - An ISO 3166 alpha-2 country code or a UN M.49 numeric-3 area code. See the Locale class description about valid country values.

Throws:

NullPointerException - thrown if either argument is null.

Locale

```
public Locale(String language)
```

Construct a locale from a language code. This constructor normalizes the language value to lowercase.

Note:

- ISO 639 is not a stable standard; some of the language codes it defines (specifically "iw", "ji", and "in") have changed. This constructor accepts both the old codes ("iw", "ji", and "in") and the new codes ("he", "yi", and "id"), but all other API on Locale will return only the OLD codes.
- For backward compatibility reasons, this constructor does not make any syntactic checks on the input.

Parameters:

language - An ISO 639 alpha-2 or alpha-3 language code, or a language subtag up to 8 characters in length. See the Locale class description about valid language values.

Throws:

NullPointerException - thrown if argument is null.

Since:

1.4

Method Detail

getDefault

```
public static Locale getDefault()
```

Gets the current value of the default locale for this instance of the Java Virtual Machine.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified. It can be changed using the setDefault method.

Returns:

the default locale for this instance of the Java Virtual Machine

getDefault

```
public static Locale getDefault(Locale.Category category)
```

Gets the current value of the default locale for the specified Category for this instance of the Java Virtual Machine.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified. It can be changed using the setDefault(Locale.Category, Locale) method.

Parameters:

category - - the specified category to get the default locale

Returns:

the default locale for the specified Category for this instance of the Java Virtual Machine

Throws:

NullPointerException - - if category is null

Since:

1.7

See Also:

`setDefault(Locale.Category, Locale)`

setDefault

```
public static void setDefault(Locale newLocale)
```

Sets the default locale for this instance of the Java Virtual Machine. This does not affect the host locale.

If there is a security manager, its `checkPermission` method is called with a `PropertyPermission("user.language", "write")` permission before the default locale is changed.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified.

Since changing the default locale may affect many different areas of functionality, this method should only be used if the caller is prepared to reinitialize locale-sensitive code running within the same Java Virtual Machine.

By setting the default locale with this method, all of the default locales for each Category are also set to the specified default locale.

Parameters:

`newLocale - the new default locale`

Throws:

`SecurityException - if a security manager exists and its checkPermission method doesn't allow the operation.`

`NullPointerException - if newLocale is null`

See Also:

`SecurityManager.checkPermission(java.security.Permission)`, `PropertyPermission`

setDefault

```
public static void setDefault(Locale.Category category,
                              Locale newLocale)
```

Sets the default locale for the specified Category for this instance of the Java Virtual Machine. This does not affect the host locale.

If there is a security manager, its checkPermission method is called with a PropertyPermission("user.language", "write") permission before the default locale is changed.

The Java Virtual Machine sets the default locale during startup based on the host environment. It is used by many locale-sensitive methods if no locale is explicitly specified.

Since changing the default locale may affect many different areas of functionality, this method should only be used if the caller is prepared to reinitialize locale-sensitive code running within the same Java Virtual Machine.

Parameters:

`category - - the specified category to set the default locale`

`newLocale - - the new default locale`

Throws:

`SecurityException - - if a security manager exists and its checkPermission method doesn't allow the operation.`

`NullPointerException - - if category and/or newLocale is null`

Since:

`1.7`

See Also:

`SecurityManager.checkPermission(java.security.Permission)`, `PropertyPermission`, `getDefault(Locale.Category)`

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all installed locales. The returned array represents the union of locales supported by the Java runtime environment and by installed `LocaleServiceProvider` implementations. It must contain at least a `Locale` instance equal to `Locale.US`.

Returns:

`An array of installed locales.`

getISOCountries

```
public static String[] getISOCountries()
```

Returns a list of all 2-letter country codes defined in ISO 3166. Can be used to create Locales.

Note: The `Locale` class also supports other codes for country (region), such as 3-letter numeric UN M.49 area codes. Therefore, the list returned by this method does not contain ALL valid codes that can be used to create Locales.

Returns:

`An array of ISO 3166 two-letter country codes.`

getISOLanguages

```
public static String[] getISOLanguages()
```

Returns a list of all 2-letter language codes defined in ISO 639. Can be used to create Locales.

Note:

- ISO 639 is not a stable standard— some languages' codes have changed. The list this function returns includes both the new and the old codes for the languages whose codes have changed.
- The `Locale` class also supports language codes up to 8 characters in length. Therefore, the list returned by this method does not contain ALL valid codes that can be used to create Locales.

Returns:

`Am array of ISO 639 two-letter language codes.`

getLanguage

```
public String getLanguage()
```

Returns the language code of this Locale.

Note: ISO 639 is not a stable standard— some languages' codes have changed. Locale's constructor recognizes both the new and the old codes for the languages whose codes have changed, but this function always returns the old code. If you want to check for a specific language whose code has changed, don't do

```
if (locale.getLanguage().equals("he")) // BAD!
   ...
```

Instead, do

```
if (locale.getLanguage().equals(new Locale("he").getLanguage()))
   ...
```

Returns:

The language code, or the empty string if none is defined.

See Also:

getDisplayLanguage()

getScript

```
public String getScript()
```

Returns the script for this locale, which should either be the empty string or an ISO 15924 4-letter script code. The first letter is uppercase and the rest are lowercase, for example, 'Latn', 'Cyrl'.

Returns:

The script code, or the empty string if none is defined.

Since:

1.7

See Also:

getDisplayScript()

getCountry

```
public String getCountry()
```

Returns the country/region code for this locale, which should either be the empty string, an uppercase ISO 3166 2-letter code, or a UN M.49 3-digit code.

Returns:

The country/region code, or the empty string if none is defined.

See Also:

getDisplayCountry()

getVariant

```
public String getVariant()
```

Returns the variant code for this locale.

Returns:

The variant code, or the empty string if none is defined.

See Also:

getDisplayVariant()

hasExtensions

```
public boolean hasExtensions()
```

Returns `true` if this `Locale` has any extensions.

Returns:

`true if this Locale has any extensions`

Since:

`1.8`

stripExtensions

```
public Locale stripExtensions()
```

Returns a copy of this `Locale` with no extensions. If this `Locale` has no extensions, this `Locale` is returned.

Returns:

`a copy of this Locale with no extensions, or this if this has no extensions`

Since:

`1.8`

getExtension

```
public String getExtension(char key)
```

Returns the extension (or private use) value associated with the specified key, or null if there is no extension associated with the key. To be well-formed, the key must be one of `[0-9A-Za-z]`. Keys are case-insensitive, so for example 'z' and 'Z' represent the same extension.

Parameters:

`key - the extension key`

Returns:

`The extension, or null if this locale defines no extension for the specified key.`

Throws:

`IllegalArgumentException - if key is not well-formed`

Since:

`1.7`

See Also:

PRIVATE_USE_EXTENSION, UNICODE_LOCALE_EXTENSION

getExtensionKeys

```
public Set<Character> getExtensionKeys()
```

Returns the set of extension keys associated with this locale, or the empty set if it has no extensions. The returned set is unmodifiable. The keys will all be lower-case.

Returns:

The set of extension keys, or the empty set if this locale has no extensions.

Since:

1.7

getUnicodeLocaleAttributes

```
public Set<String> getUnicodeLocaleAttributes()
```

Returns the set of unicode locale attributes associated with this locale, or the empty set if it has no attributes. The returned set is unmodifiable.

Returns:

The set of attributes.

Since:

1.7

getUnicodeLocaleType

```
public String getUnicodeLocaleType(String key)
```

Returns the Unicode locale type associated with the specified Unicode locale key for this locale. Returns the empty string for keys that are defined with no type. Returns null if the key is not defined. Keys are case-insensitive. The key must be two alphanumeric characters ([0-9a-zA-Z]), or an IllegalArgumentException is thrown.

Parameters:

key - the Unicode locale key

Returns:

The Unicode locale type associated with the key, or null if the locale does not define the key.

Throws:

IllegalArgumentException - if the key is not well-formed

NullPointerException - if key is null

Since:

1.7

getUnicodeLocaleKeys

```
public Set<String> getUnicodeLocaleKeys()
```

Returns the set of Unicode locale keys defined by this locale, or the empty set if this locale has none. The returned set is immutable. Keys are all lower case.

Returns:

The set of Unicode locale keys, or the empty set if this locale has no Unicode locale keywords.

Since:

1.7

toString

```
public final String toString()
```

Returns a string representation of this `Locale` object, consisting of language, country, variant, script, and extensions as below:

> language + "_" + country + "_" + (variant + "_#" | "#") + script + "-" + extensions

Language is always lower case, country is always upper case, script is always title case, and extensions are always lower case. Extensions and private use subtags will be in canonical order as explained in `toLanguageTag()`.

When the locale has neither script nor extensions, the result is the same as in Java 6 and prior.

If both the language and country fields are missing, this function will return the empty string, even if the variant, script, or extensions field is present (you can't have a locale with just a variant, the variant must accompany a well-formed language or country code).

If script or extensions are present and variant is missing, no underscore is added before the "#".

This behavior is designed to support debugging and to be compatible with previous uses of `toString` that expected language, country, and variant fields

only. To represent a Locale as a String for interchange purposes, use `toLanguageTag()`.

Examples:

- `en`
- `de_DE`
- `_GB`
- `en_US_WIN`
- `de__POSIX`
- `zh_CN_#Hans`
- `zh_TW_#Hant-x-java`
- `th_TH_TH_#u-nu-thai`

Overrides:

`toString` in class `Object`

Returns:

`A string representation of the Locale, for debugging.`

See Also:

`getDisplayName()`, `toLanguageTag()`

toLanguageTag

```
public String toLanguageTag()
```

Returns a well-formed IETF BCP 47 language tag representing this locale.

If this `Locale` has a language, country, or variant that does not satisfy the IETF BCP 47 language tag syntax requirements, this method handles these fields as described below:

Language: If language is empty, or not well-formed (for example "a" or "e2"), it will be emitted as "und" (Undetermined).

Country: If country is not well-formed (for example "12" or "USA"), it will be omitted.

Variant: If variant **is** well-formed, each sub-segment (delimited by '-' or '_') is emitted as a subtag. Otherwise:

- if all sub-segments match `[0-9a-zA-Z]{1,8}` (for example "WIN" or "Oracle_JDK_Standard_Edition"), the first ill-formed sub-segment and all following will be appended to the private use subtag. The first appended subtag will be "lvariant", followed by the sub-segments in order, separated by hyphen. For example, "x-lvariant-WIN", "Oracle-x-lvariant-JDK-Standard-Edition".
- if any sub-segment does not match `[0-9a-zA-Z]{1,8}`, the variant will be truncated and the problematic sub-segment and all following sub-segments will be omitted. If the remainder is non-empty, it will be

emitted as a private use subtag as above (even if the remainder turns out to be well-formed). For example, "Solaris_isjustthecoolestthing" is emitted as "x-lvariant-Solaris", not as "solaris".

Special Conversions: Java supports some old locale representations, including deprecated ISO language codes, for compatibility. This method performs the following conversions:

- Deprecated ISO language codes "iw", "ji", and "in" are converted to "he", "yi", and "id", respectively.
- A locale with language "no", country "NO", and variant "NY", representing Norwegian Nynorsk (Norway), is converted to a language tag "nn-NO".

Note: Although the language tag created by this method is well-formed (satisfies the syntax requirements defined by the IETF BCP 47 specification), it is not necessarily a valid BCP 47 language tag. For example,

```
new Locale("xx", "YY").toLanguageTag();
```

will return "xx-YY", but the language subtag "xx" and the region subtag "YY" are invalid because they are not registered in the IANA Language Subtag Registry.

Returns:

a BCP47 language tag representing the locale

Since:

1.7

See Also:

`forLanguageTag(String)`

forLanguageTag

```
public static Locale forLanguageTag(String languageTag)
```

Returns a locale for the specified IETF BCP 47 language tag string.

If the specified language tag contains any ill-formed subtags, the first such subtag and all following subtags are ignored. Compare to `Locale.Builder.setLanguageTag(java.lang.String)` which throws an exception in this case.

The following **conversions** are performed:

- The language code "und" is mapped to language "".
- The language codes "he", "yi", and "id" are mapped to "iw", "ji", and "in" respectively. (This is the same canonicalization that's done in Locale's constructors.)

- The portion of a private use subtag prefixed by "lvariant", if any, is removed and appended to the variant field in the result locale (without case normalization). If it is then empty, the private use subtag is discarded:

```
Locale loc;
loc = Locale.forLanguageTag("en-US-x-lvariant-POSIX");
loc.getVariant(); // returns "POSIX"
loc.getExtension('x'); // returns null

loc = Locale.forLanguageTag("de-POSIX-x-URP-lvariant-Abc-Def");
loc.getVariant(); // returns "POSIX_Abc_Def"
loc.getExtension('x'); // returns "urp"
```

- When the languageTag argument contains an extlang subtag, the first such subtag is used as the language, and the primary language subtag and other extlang subtags are ignored:

```
Locale.forLanguageTag("ar-aao").getLanguage(); // returns "aao"
Locale.forLanguageTag("en-abc-def-us").toString(); // returns "abc_US"
```

- Case is normalized except for variant tags, which are left unchanged. Language is normalized to lower case, script to title case, country to upper case, and extensions to lower case.
- If, after processing, the locale would exactly match either ja_JP_JP or th_TH_TH with no extensions, the appropriate extensions are added as though the constructor had been called:

```
Locale.forLanguageTag("ja-JP-x-lvariant-JP").toLanguageTag();
// returns "ja-JP-u-ca-japanese-x-lvariant-JP"
Locale.forLanguageTag("th-TH-x-lvariant-TH").toLanguageTag();
// returns "th-TH-u-nu-thai-x-lvariant-TH"
```

This implements the 'Language-Tag' production of BCP47, and so supports grandfathered (regular and irregular) as well as private use language tags. Stand alone private use tags are represented as empty language and extension 'x-whatever', and grandfathered tags are converted to their canonical replacements where they exist.

Grandfathered tags with canonical replacements are as follows:

grandfathered tag	**modern replacement**
art-lojban	jbo
i-ami	ami
i-bnn	bnn
i-hak	hak
i-klingon	tlh

i-lux	lb
i-navajo	nv
i-pwn	pwn
i-tao	tao
i-tay	tay
i-tsu	tsu
no-bok	nb
no-nyn	nn
sgn-BE-FR	sfb
sgn-BE-NL	vgt
sgn-CH-DE	sgg
zh-guoyu	cmn
zh-hakka	hak
zh-min-nan	nan
zh-xiang	hsn

Grandfathered tags with no modern replacement will be converted as follows:

grandfathered tag	**converts to**
cel-gaulish	xtg-x-cel-gaulish
en-GB-oed	en-GB-x-oed
i-default	en-x-i-default
i-enochian	und-x-i-enochian
i-mingo	see-x-i-mingo
zh-min	nan-x-zh-min

For a list of all grandfathered tags, see the IANA Language Subtag Registry (search for "Type: grandfathered").

Note: there is no guarantee that `toLanguageTag` and `forLanguageTag` will round-trip.

Parameters:

`languageTag - the language tag`

Returns:

`The locale that best represents the language tag.`

Throws:

`NullPointerException - if languageTag is null`

Since:

`1.7`

See Also:

`toLanguageTag(), Locale.Builder.setLanguageTag(String)`

getISO3Language

```
public String getISO3Language()
                         throws MissingResourceException
```

Returns a three-letter abbreviation of this locale's language. If the language matches an ISO 639-1 two-letter code, the corresponding ISO 639-2/T three-letter lowercase code is returned. The ISO 639-2 language codes can be found on-line, see "Codes for the Representation of Names of Languages Part 2: Alpha-3 Code". If the locale specifies a three-letter language, the language is returned as is. If the locale does not specify a language the empty string is returned.

Returns:

A three-letter abbreviation of this locale's language.

Throws:

MissingResourceException - Throws MissingResourceException if three-letter language abbreviation is not available for this locale.

getISO3Country

```
public String getISO3Country()
                       throws MissingResourceException
```

Returns a three-letter abbreviation for this locale's country. If the country matches an ISO 3166-1 alpha-2 code, the corresponding ISO 3166-1 alpha-3 uppercase code is returned. If the locale doesn't specify a country, this will be the empty string.

The ISO 3166-1 codes can be found on-line.

Returns:

A three-letter abbreviation of this locale's country.

Throws:

MissingResourceException - Throws MissingResourceException if the three-letter country abbreviation is not available for this locale.

getDisplayLanguage

```
public final String getDisplayLanguage()
```

Returns a name for the locale's language that is appropriate for display to the user. If possible, the name returned will be localized for the default DISPLAY locale. For example, if the locale is fr_FR and the default DISPLAY

locale is en_US, getDisplayLanguage() will return "French"; if the locale is en_US and the default DISPLAY locale is fr_FR, getDisplayLanguage() will return "anglais". If the name returned cannot be localized for the default DISPLAY locale, (say, we don't have a Japanese name for Croatian), this function falls back on the English name, and uses the ISO code as a last-resort value. If the locale doesn't specify a language, this function returns the empty string.

Returns:

The name of the display language.

getDisplayLanguage

```
public String getDisplayLanguage(Locale inLocale)
```

Returns a name for the locale's language that is appropriate for display to the user. If possible, the name returned will be localized according to inLocale. For example, if the locale is fr_FR and inLocale is en_US, getDisplayLanguage() will return "French"; if the locale is en_US and inLocale is fr_FR, getDisplayLanguage() will return "anglais". If the name returned cannot be localized according to inLocale, (say, we don't have a Japanese name for Croatian), this function falls back on the English name, and finally on the ISO code as a last-resort value. If the locale doesn't specify a language, this function returns the empty string.

Parameters:

inLocale - The locale for which to retrieve the display language.

Returns:

The name of the display language appropriate to the given locale.

Throws:

NullPointerException - if inLocale is null

getDisplayScript

```
public String getDisplayScript()
```

Returns a name for the the locale's script that is appropriate for display to the user. If possible, the name will be localized for the default DISPLAY locale. Returns the empty string if this locale doesn't specify a script code.

Returns:

the display name of the script code for the current default DISPLAY locale

Since:

1.7

getDisplayScript

```
public String getDisplayScript(Locale inLocale)
```

Returns a name for the locale's script that is appropriate for display to the user. If possible, the name will be localized for the given locale. Returns the empty string if this locale doesn't specify a script code.

Parameters:

`inLocale - The locale for which to retrieve the display script.`

Returns:

`the display name of the script code for the current default DISPLAY locale`

Throws:

`NullPointerException - if inLocale is null`

Since:

`1.7`

getDisplayCountry

```
public final String getDisplayCountry()
```

Returns a name for the locale's country that is appropriate for display to the user. If possible, the name returned will be localized for the default DISPLAY locale. For example, if the locale is fr_FR and the default DISPLAY locale is en_US, getDisplayCountry() will return "France"; if the locale is en_US and the default DISPLAY locale is fr_FR, getDisplayCountry() will return "Etats-Unis". If the name returned cannot be localized for the default DISPLAY locale, (say, we don't have a Japanese name for Croatia), this function falls back on the English name, and uses the ISO code as a last-resort value. If the locale doesn't specify a country, this function returns the empty string.

Returns:

`The name of the country appropriate to the locale.`

getDisplayCountry

```
public String getDisplayCountry(Locale inLocale)
```

Returns a name for the locale's country that is appropriate for display to the user. If possible, the name returned will be localized according to inLocale. For example, if the locale is fr_FR and inLocale is en_US, getDisplayCountry() will return "France"; if the locale is en_US and inLocale is fr_FR, getDisplayCountry() will return "Etats-Unis". If the name returned cannot be localized according to inLocale. (say, we don't have a Japanese

name for Croatia), this function falls back on the English name, and finally on the ISO code as a last-resort value. If the locale doesn't specify a country, this function returns the empty string.

Parameters:

`inLocale - The locale for which to retrieve the display country.`

Returns:

`The name of the country appropriate to the given locale.`

Throws:

`NullPointerException - if inLocale is null`

getDisplayVariant

```
public final String getDisplayVariant()
```

Returns a name for the locale's variant code that is appropriate for display to the user. If possible, the name will be localized for the default `DISPLAY` locale. If the locale doesn't specify a variant code, this function returns the empty string.

Returns:

`The name of the display variant code appropriate to the locale.`

getDisplayVariant

```
public String getDisplayVariant(Locale inLocale)
```

Returns a name for the locale's variant code that is appropriate for display to the user. If possible, the name will be localized for inLocale. If the locale doesn't specify a variant code, this function returns the empty string.

Parameters:

`inLocale - The locale for which to retrieve the display variant code.`

Returns:

`The name of the display variant code appropriate to the given locale.`

Throws:

`NullPointerException - if inLocale is null`

getDisplayName

```
public final String getDisplayName()
```

Returns a name for the locale that is appropriate for display to the user. This will be the values returned by getDisplayLanguage(), getDisplayScript(), getDisplayCountry(), and getDisplayVariant() assembled into a single string. The the non-empty values are used in order, with the second and subsequent names in parentheses. For example:

language (script, country, variant)
language (country)
language (variant)
script (country)
country

depending on which fields are specified in the locale. If the language, script, country, and variant fields are all empty, this function returns the empty string.

Returns:

`The name of the locale appropriate to display.`

getDisplayName

```
public String getDisplayName(Locale inLocale)
```

Returns a name for the locale that is appropriate for display to the user. This will be the values returned by getDisplayLanguage(), getDisplayScript(),getDisplayCountry(), and getDisplayVariant() assembled into a single string. The non-empty values are used in order, with the second and subsequent names in parentheses. For example:

language (script, country, variant)
language (country)
language (variant)
script (country)
country

depending on which fields are specified in the locale. If the language, script, country, and variant fields are all empty, this function returns the empty string.

Parameters:

`inLocale - The locale for which to retrieve the display name.`

Returns:

`The name of the locale appropriate to display.`

Throws:

`NullPointerException - if inLocale is null`

clone

```
public Object clone()
```

Overrides Cloneable.

Overrides:

`clone` in class `Object`

Returns:

a clone of this instance.

See Also:

`Cloneable`

hashCode

```
public int hashCode()
```

Override hashCode. Since Locales are often used in hashtables, caches the value for speed.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

equals

```
public boolean equals(Object obj)
```

Returns true if this Locale is equal to another object. A Locale is deemed equal to another Locale with identical language, script, country, variant and extensions, and unequal to all other objects.

Overrides:

`equals` in class `Object`

Parameters:

obj - the reference object with which to compare.

Returns:

true if this Locale is equal to the specified object.

See Also:

Object.hashCode(), HashMap

filter

```
public static List<Locale> filter(List<Locale.LanguageRange> priorityList,
                                  Collection<Locale> locales,
                                  Locale.FilteringMode mode)
```

Returns a list of matching Locale instances using the filtering mechanism defined in RFC 4647.

Parameters:

priorityList - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

locales - Locale instances used for matching

mode - filtering mode

Returns:

a list of Locale instances for matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

Throws:

NullPointerException - if priorityList or locales is null

IllegalArgumentException - if one or more extended language ranges are included in the given list when Locale.FilteringMode.REJECT_EXTENDED_RANGES is specified

Since:

1.8

filter

```
public static List<Locale> filter(List<Locale.LanguageRange> priorityList,
                                  Collection<Locale> locales)
```

Returns a list of matching Locale instances using the filtering mechanism defined in RFC 4647. This is equivalent to filter(List, Collection, FilteringMode) when mode is Locale.FilteringMode.AUTOSELECT_FILTERING.

Parameters:

priorityList - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

locales - Locale instances used for matching

Returns:

a list of Locale instances for matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

Throws:

NullPointerException - if priorityList or locales is null

Since:

1.8

filterTags

```
public static List<String> filterTags(List<Locale.LanguageRange> priorityList,
                                      Collection<String> tags,
                                      Locale.FilteringMode mode)
```

Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647.

Parameters:

priorityList - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

tags - language tags

mode - filtering mode

Returns:

a list of matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

Throws:

NullPointerException - if priorityList or tags is null

IllegalArgumentException - if one or more extended language ranges are included in the given list when Locale.FilteringMode.REJECT_EXTENDED_RANGES is specified

Since:

1.8

filterTags

```
public static List<String> filterTags(List<Locale.LanguageRange> priorityList,
                                      Collection<String> tags)
```

Returns a list of matching languages tags using the basic filtering mechanism defined in RFC 4647. This is equivalent to `filterTags(List, Collection, FilteringMode)` when `mode` is `Locale.FilteringMode.AUTOSELECT_FILTERING`.

Parameters:

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`tags` - language tags

Returns:

a list of matching language tags sorted in descending order based on priority or weight, or an empty list if nothing matches. The list is modifiable.

Throws:

`NullPointerException` - if priorityList or tags is null

Since:

1.8

lookup

```
public static Locale lookup(List<Locale.LanguageRange> priorityList,
                            Collection<Locale> locales)
```

Returns a `Locale` instance for the best-matching language tag using the lookup mechanism defined in RFC 4647.

Parameters:

`priorityList` - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight

`locales` - Locale instances used for matching

Returns:

the best matching Locale instance chosen based on priority or weight, or null if nothing matches.

Throws:

`NullPointerException` - if priorityList or tags is null

Since:

1.8

lookupTag

```
public static String lookupTag(List<Locale.LanguageRange> priorityList,
                               Collection<String> tags)
```

Returns the best-matching language tag using the lookup mechanism defined in RFC 4647.

Parameters:

`priorityList - user's Language Priority List in which each language tag is sorted in descending order based on priority or weight`

`tags - language tangs used for matching`

Returns:

`the best matching language tag chosen based on priority or weight, or null if nothing matches.`

Throws:

`NullPointerException - if priorityList or tags is null`

Since:

`1.8`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.